

Assignment 2: Data Modelling

Danial Ansari (s4119075); COSC2670

Data preparation and sampling. The full dataset (13,611 beans, 16 numeric inputs, 7 classes) contains no missing or non-positive values but has 68 exact duplicate rows, which were removed so that the same bean cannot appear twice in a sample (e.g. in both a training and a test set). From the remaining 13,543 complete, unique rows, three simple random samples without replacement were drawn with a fixed seed (300, 2,000 and 2,000 rows) and saved as COSC2670-s4119075-A2SampleOne/Two/Three.csv. Each task reloads its saved file, so every result in a task is based on the same sample. All analysis uses Python (pandas, scikit-learn; Pedregosa et al., 2011).

Task 1: Regression

1.1 Relationship between MajorAxisLength and Perimeter

Figure 1 shows a very strong, positive and almost linear relationship: longer beans have longer outlines (Pearson $r = 0.977$, Spearman $\rho = 0.970$). Several patterns and unusual observations stand out:

- **Distinct Bombay group.** The 13 Bombay beans form a separate cluster in the top-right corner (MajorAxisLength 526–680 px, Perimeter 1,430–1,777 px). They are the only IQR outliers on both variables, but they are genuine large beans that follow the same trend, so they were kept. There is a gap with no beans between about 450 and 525 px, so the data are not spread evenly over the range, and this small group strongly influences (has high leverage on) the fitted line.
- **Class-specific bands.** At the same length, round classes have longer perimeters than elongated ones. Seker and Barbunya lie above the main trend, and Horoz and Cali lie below it. The relationship therefore also depends on bean shape (width), not only on length.
- **Increasing spread.** The vertical scatter grows with size and is largest for Barbunya (e.g. one bean of about 405 px has a perimeter of 1,340 px), which suggests irregular outlines and mild heteroscedasticity, meaning the variance is not constant.

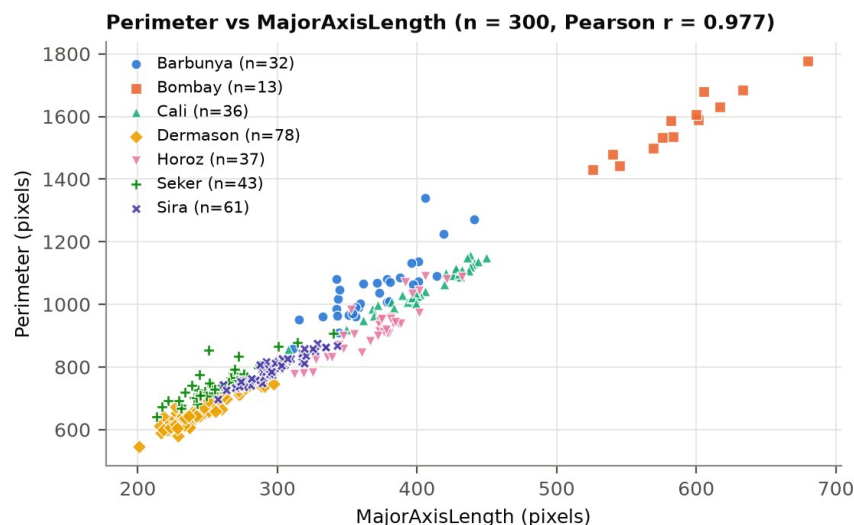


Figure 1. Perimeter versus MajorAxisLength for the 300 sampled beans; colour and marker show the bean class, which is used only to explain the patterns.

1.2 Simple linear regression model

The sample was split at random into 240 training and 60 test beans. An ordinary least-squares model with Perimeter as the dependent variable and MajorAxisLength as the independent variable was fitted on the training data:

$$\text{Perimeter} = 53.90 + 2.520 \times \text{MajorAxisLength} \quad (95\% \text{ CI: slope } [2.447, 2.593], \text{ intercept } [29.5, 78.3])$$

Interpretation of the coefficients. The slope means that every extra pixel of major-axis length increases the expected perimeter by about 2.52 pixels, or about 25 px per 10 px of length. This is geometrically plausible. An ellipse with major axis L has a perimeter between $2L$ (a very flat shape) and $\pi L \approx 3.14L$ (a circle), and beans are ellipse-like with an average aspect ratio of about 1.6, so a factor near 2.5 is expected. The intercept is the

predicted perimeter for a bean of zero length. That lies far outside the observed range (201–680 px), so it has no physical meaning and only positions the line. It is positive because perimeter also depends on the minor axis (width), which this model does not include.

Table 1. Regression performance (Perimeter in pixels).

Data	R ²	RMSE	MAE	MAPE
Training set (n = 240)	0.951	48.95	35.35	3.98%
Test set (n = 60)	0.971	34.60	27.29	3.40%
10-fold CV, all 300 (mean ± SD)	0.953 ± 0.019	45.11 ± 12.68	—	—

How well does the model represent the relationship? Using R² (the share of variance explained), RMSE and MAE (typical errors in pixels) and MAPE (relative error), the line explains about 95–97% of the variation in perimeter, with a typical error of 27–35 px (3.4–4.0%). The test R² is slightly higher than the training R² only because the small random test set happens to contain few of the irregular Barbunya beans. 10-fold cross-validation confirms a stable R² of 0.953. The residual plot (Figure 2b) shows the model's limitation: the residuals are **not random but depend on the class**. Elongated Horoz (mean residual –61 px) and Cali (–26 px) are over-predicted, while round Seker (+45 px), Barbunya (+59 px) and Bombay (+37 px) are under-predicted. The residuals correlate strongly with AspectRatio ($r = -0.72$), so most of the remaining error comes from bean shape, which a single length variable cannot capture. Overall, the linear model is an accurate first-order description of how perimeter grows with size. Adding a width or shape variable (multiple regression), or letting the slope differ by class, would remove the systematic error.

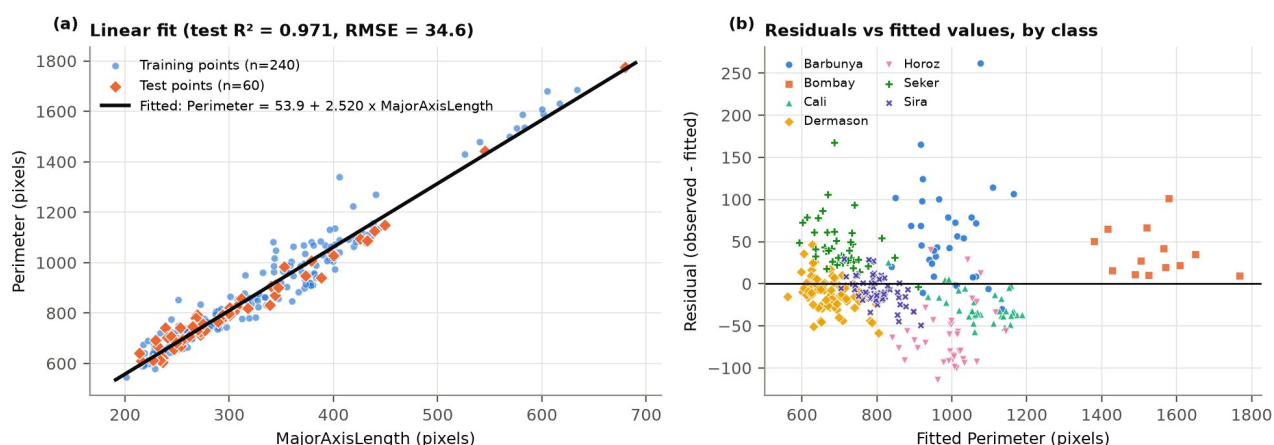


Figure 2. (a) Training and test points with the fitted regression line; (b) residuals versus fitted values coloured by class, showing systematic, shape-related errors.

Task 2: Classification

Sample Two (2,000 beans: Dermason 546, Sira 400, Horoz 286, Seker 273, Cali 225, Barbunya 190, Bombay 80) was split once into a **stratified 80/20 training/test split** (1,600/400). Stratification keeps the class proportions, including the small Bombay class, the same in both sets, and the same split is reused for every classifier. All model development (preprocessing parameters and hyperparameters) used only the training data, with **10-fold stratified cross-validation repeated 3 times (30 folds)**. Preprocessing was placed inside a scikit-learn Pipeline so that it is re-fitted on the training folds only, which prevents data leakage. Because the classes are imbalanced, the selection criterion is **macro-F1**, which weights every class equally. The test set was used once per model for the final evaluation with the same metrics: accuracy, macro precision, macro recall, macro F1 and the confusion matrix.

2.1 k-Nearest Neighbours (kNN)

Preprocessing. kNN classifies a bean by the classes of the beans closest to it in Euclidean distance, so features with large numeric ranges dominate. Here the scales differ by about seven orders of magnitude (Area $\approx 53,000$ px versus ShapeFactor2 ≈ 0.0017). All 16 inputs were therefore z-standardised to mean 0 and SD 1. The evidence is clear: without scaling, the best CV macro-F1 is only 0.631, against 0.931 with scaling. There were no missing values or categorical inputs to handle.

Choice of k. $k = 1$ –40 was evaluated (Figure 3a). Very small k overfits to individual noisy beans ($k = 1$: macro-F1 0.907; $k = 2$: 0.894, caused by frequent ties). Performance then reaches a broad plateau between $k \approx 8$ and 20 (all within 0.005 of the best), and declines slowly for larger k (0.922 at $k = 40$), as neighbourhoods become so large that boundary beans are absorbed by bigger neighbouring classes. **$k = 14$** gave the highest CV macro-

F1 (0.931 ± 0.017 ; accuracy 0.919). It lies in the middle of the plateau, so the choice is robust and is not a single lucky peak.

Test performance. Accuracy = 0.923, macro precision = 0.931, macro recall = 0.927 and macro F1 = 0.929. Bombay is recognised perfectly (it is far larger than every other variety). Most of the 31 errors involve two pairs of morphologically similar varieties: **Dermason** ↔ **Sira** (13 errors) and **Barbunya** ↔ **Cali** (7 errors), as shown in Figure 3b. This matches the original study of the dataset, in which Sira was also the hardest class (Koklu & Ozkan, 2020).

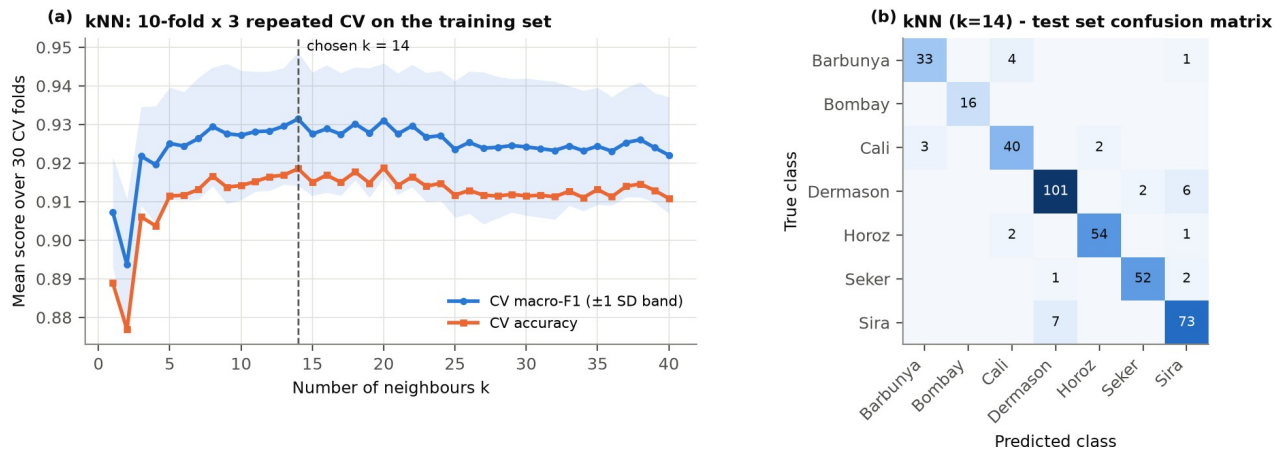


Figure 3. (a) Repeated 10-fold CV macro-F1 (± 1 SD band) and accuracy for $k = 1-40$ on the training set; (b) confusion matrix of the selected kNN ($k = 14$) on the 400 test beans.

2.2B Advanced improvement strategy: Local Mean-based kNN

Motivation. Standard kNN has three weaknesses on this data. (i) The majority vote favours large, dense classes: near the Dermason/Sira boundary most neighbours come from the bigger Dermason class (546 vs 400 beans). (ii) A few noisy or atypical neighbours (e.g. irregular Barbunya beans) can flip the vote. (iii) Votes ignore how close the neighbours are and can produce ties.

Modification of the prediction mechanism. I implemented, from scratch as a scikit-learn estimator, the *local mean-based kNN* rule (LMKNN; Mitani & Hamamoto, 2006), which changes how neighbours contribute to the decision:

- **Class-conditional neighbours.** For a query bean x , the k nearest neighbours are searched separately *within each class*, so every class gets exactly k representatives regardless of its size or local density.
- **Local means instead of votes.** The k neighbours of class c are averaged into a local mean vector $m_c(x) = (1/k) \cdot \sum x_i^c$. This acts as a local class centroid that adapts to where the query lies, and averaging cancels the effect of single noisy neighbours.
- **Distance-based decision.** x is assigned to the class whose local mean is closest, $\hat{y} = \operatorname{argmin}_c \|x - m_c(x)\|$. The decision uses continuous distances, so voting ties disappear. With $k = 1$ the rule equals 1-NN, and as k approaches the class size it becomes a nearest-centroid classifier, so k gives a smooth bias–variance trade-off.

I also implemented a multi-scale extension, the local mean-based pseudo nearest neighbour (LMPNN; Gou et al., 2014). It computes the local means of the first 1, 2, ..., k neighbours of each class and scores each class by the weighted sum $\sum_j (1/j) \cdot \|x - m_c^j(x)\|$. Both variants use the same standardisation pipeline, CV folds, k range (1–40), selection metric and test split as the original kNN.

Table 2. Classification results. CV = repeated 10-fold CV on the training set; test = 400 held-out beans.

Model	Chosen parameter(s)	CV macro-F1	Test acc.	Test macro-P	Test macro-R	Test macro-F1
kNN (original, 2.1)	$k = 14$	0.931	0.923	0.931	0.927	0.929
LMKNN (proposed, 2.2B)	$k = 9$	0.936	0.935	0.941	0.935	0.938
LMPNN (extension)	$k = 38$	0.934	0.928	0.937	0.933	0.935
Decision Tree (2.3)	entropy, depth 6, leaf 10	0.907	0.873	0.880	0.879	0.879

Results. LMKNN obtained the best CV macro-F1 (0.936 at $k = 9$) and was selected as the proposed model. It also improved every test metric over the original kNN: accuracy rose from 0.923 to 0.935 (26 instead of 31 errors) and macro-F1 from 0.929 to 0.938 (Table 2, Figure 4b). The per-class view (Figure 4c) shows that the

gains are concentrated where the motivation predicted, in the overlapping small-bean group of different sizes: Dermason F1 0.927 → 0.950, Sira 0.896 → 0.915, Seker 0.954 → 0.972. Sira → Dermason errors fell from 7 to 5, Dermason → Sira from 6 to 5, and Dermason → Seker from 2 to 0. Figure 4a also shows the different mechanisms. The original kNN degrades for larger k , as large classes increasingly dominate the vote. LMKNN stays above kNN for all $k \geq 5$, and LMPNN rises steadily and is the least sensitive to large k , because its $1/k$ weights keep the nearest neighbours dominant.

Does it improve performance? Yes, consistently but modestly. The improvement appears both in CV (+0.45 pp macro-F1) and on the test set (+1.25 pp accuracy). However, an exact McNemar test on the paired test predictions (7 beans corrected only by LMKNN versus 2 only by kNN) gives $p = 0.18$, so the difference is not statistically significant with 400 test beans. The modification helps when classes overlap and differ in size or density, because a local centroid per class estimates the class-conditional structure around x better than a vote dominated by the larger class. It cannot help Barbunya and Cali (F1 unchanged at 0.892 and 0.879), whose overlap is genuine in this feature space (Figure 8a): no rule based on Euclidean distance separates them. Too large a k pulls the local mean across curved class boundaries (bias), which is why the best k is smaller than for kNN. The cost is small: prediction needs one neighbour search per class (14 ms vs 9 ms for 400 beans). LMKNN still depends on scaling and on a Euclidean metric in which the many correlated size features are over-weighted, which also limits the gain.

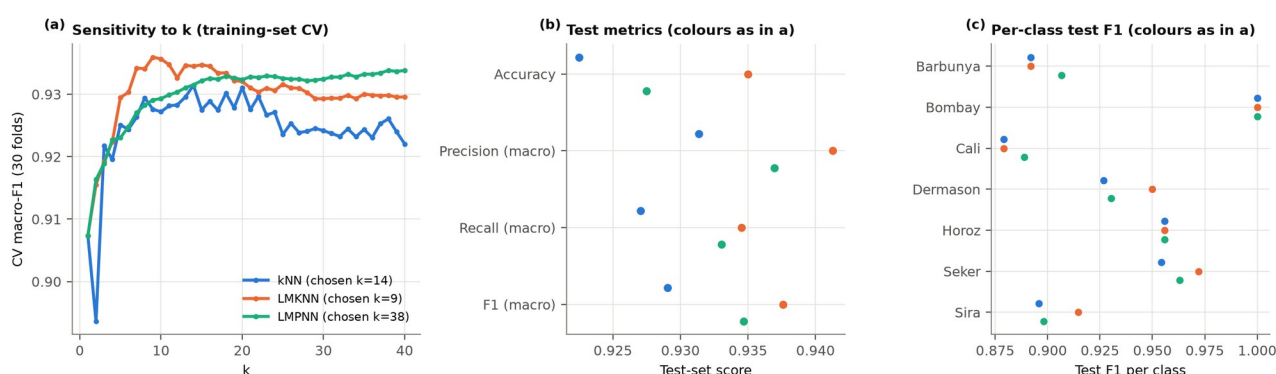


Figure 4. (a) CV macro-F1 versus k for the original kNN and the two local-mean modifications; (b) test-set metrics on the same split; (c) per-class test F1. Colours are consistent across panels.

2.3 Decision Tree

A decision tree splits on one feature threshold at a time, so it is scale-invariant and needs no standardisation. Its key complexity parameters were tuned jointly with the same 30-fold CV and macro-F1 criterion: **max_depth** (1–20 and unlimited), **min_samples_leaf** (1, 2, 5, 10, 20) and the split **criterion** (Gini or entropy), 210 combinations in total. The best tree uses **entropy**, **max_depth = 6** and **min_samples_leaf = 10** (CV macro-F1 0.907; the final tree has 38 leaves). The validation curve (Figure 5a) justifies this choice. Trees with depth ≤ 4 underfit (depth 3: F1 0.75). The validation score peaks at depth 6 and then stays flat or falls slightly while the training score keeps rising, so deeper trees only add complexity. An unpruned tree reaches a training F1 of 1.000 but a CV F1 of only 0.895, which is clear overfitting. A minimum of 10 beans per leaf stops leaves from memorising individual beans.

Comparison with the original kNN. On the test set the tree reaches accuracy 0.873, macro precision 0.880, recall 0.879 and F1 0.879, about 5 percentage points below kNN on every metric (Table 2, Figure 5b). The CV gap is smaller (0.907 vs 0.931), so part of the test gap is sampling noise, but kNN is better on both. The tree's weakest classes are Barbunya (F1 0.73; 9 of 38 predicted as Cali), Sira (0.82) and Cali (0.83). Bombay (1.00) and Seker (0.96) remain easy. The feature importances (Figure 5c) show that the tree first uses size (MajorAxisLength 0.33, MinorAxisLength 0.15) and then shape (ShapeFactor1 0.16, ShapeFactor3 0.09). Redundant copies of size, such as ConvexArea and EquivDiameter, receive zero importance because the tree only needs one variable from each correlated group.

Strengths and weaknesses of each model on this dataset:

- **Predictive performance.** kNN wins. The boundaries between varieties are smooth, oblique combinations of correlated size and shape features (e.g. Dermason versus Sira lie along a size–shape diagonal). A tree can only approximate such boundaries with axis-parallel 'staircases', which needs many splits and therefore more data, whereas kNN adapts locally.
- **Interpretability.** The tree wins. Its 38 if–then rules and feature importances explain *why* a bean is classified (e.g. Bombay is isolated by a single size threshold). kNN has no global model; it can only justify a prediction by pointing to similar beans.

- **Model complexity and cost.** kNN is a lazy learner. It has almost no training cost, but it must store all 1,600 training beans and compute distances to them for every prediction. The tree is a compact model of depth 6, slower to train (23 ms vs 7 ms) but faster to predict (2.5 ms vs 5.5 ms), and it scales better to large data.
- **Sensitivity to data characteristics.** kNN is very sensitive to feature scaling (0.63 vs 0.93), to redundant or irrelevant features (here eight correlated size variables share the distance), to noise when k is small, and to class imbalance. The tree is insensitive to scaling and monotone transformations and ignores irrelevant features, but it has high variance: small changes in the data can change the splits, and it overfits without depth or leaf constraints. Its leaves give little support to small classes.

Overall, kNN (and its local-mean modification) is preferable when accuracy matters, and the tree is preferable when an explainable rule set is required. Tree ensembles such as random forests would reduce the tree's variance and narrow the accuracy gap.

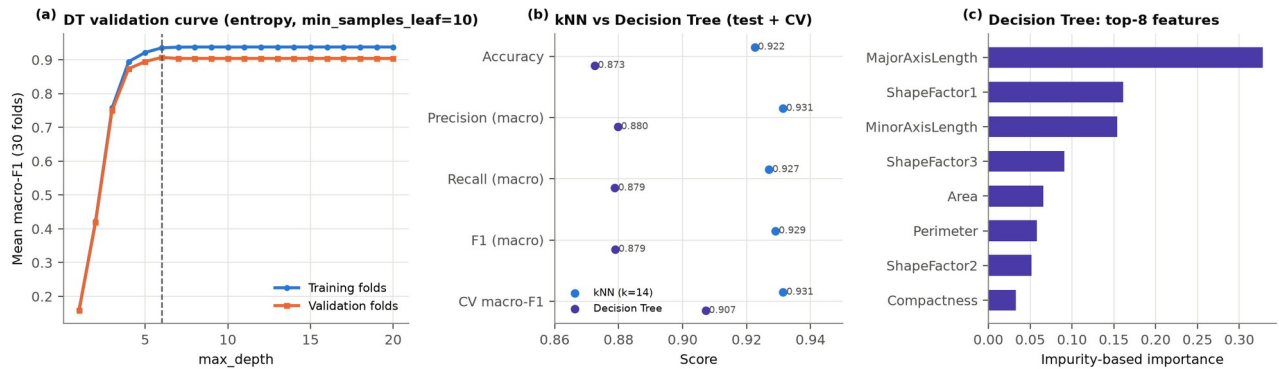


Figure 5. (a) Training versus validation macro-F1 over max_depth for the best criterion and leaf size; (b) kNN versus Decision Tree on test metrics and CV macro-F1; (c) the eight most important tree features.

Task 3: Clustering

Sample Three (2,000 beans) was used with all 16 inputs. The Class column was removed before any modelling. It was **never used to fit a model or choose a parameter**, only afterwards for external evaluation (adjusted Rand index, ARI; normalised mutual information, NMI; homogeneity; completeness; contingency tables). The inputs were z-standardised, because distance-based clustering would otherwise be driven by Area and ConvexArea. Internal quality was measured with the silhouette coefficient (higher is better; Rousseeuw, 1987), the Davies–Bouldin index (lower is better) and the Calinski–Harabasz index (higher is better). ARI is 0 for random labels and 1 for perfect agreement (Hubert & Arabie, 1985). Note that 21 of the 120 feature pairs have $|r| > 0.9$.

3.1 k-Means

k-Means (k-means++ initialisation, 10 restarts) was run for $k = 2$ –12 (Figure 6). The criteria do not fully agree. The elbow of the inertia curve (the point farthest from the chord between the first and last points, a 'kneedle'-style rule; Satopää et al., 2011) is $k = 5$. The silhouette is highest at $k = 3$ (0.403), Davies–Bouldin is lowest at $k = 3$ (0.910), and Calinski–Harabasz is highest at $k = 2$. Taking the majority of these internal criteria, $k = 3$ was chosen. It is a clear joint optimum of the two separation-based indices, while the inertia curve declines smoothly with only a weak elbow.

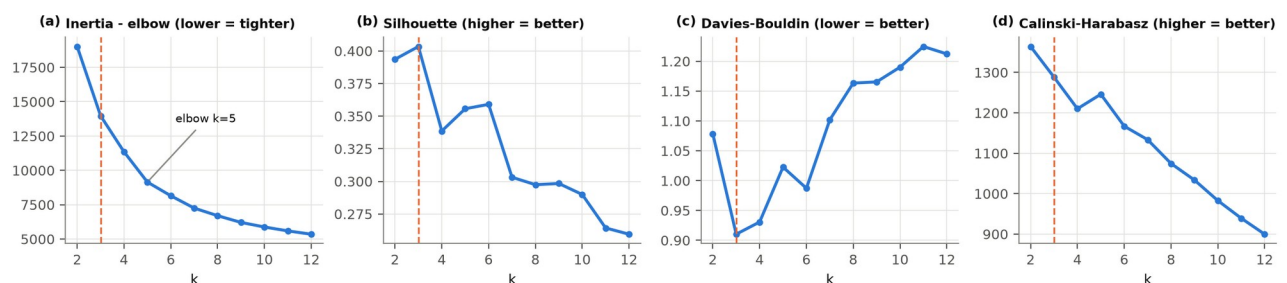


Figure 6. Effect of the number of clusters k on (a) inertia, (b) silhouette, (c) Davies–Bouldin and (d) Calinski–Harabasz for k-Means; the dashed line marks the chosen $k = 3$.

Table 3. *k*-Means (*k* = 3) clusters versus the true classes (contingency table) and mean cluster profiles.

Cluster	Barbunya	Bombay	Cali	Dermason	Horoz	Seker	Sira	Mean Area	Aspect ratio
0 (n = 820)	190	0	247	5	274	0	104	64,036	1.78
1 (n = 1,098)	17	0	0	525	4	295	257	37,237	1.43
2 (n = 82)	1	81	0	0	0	0	0	172,026	1.59

Results and interpretation. The *k* = 3 solution has silhouette 0.403, DB 0.910 and CH 1,288. Against the varieties it reaches ARI 0.306 and NMI 0.495, with high completeness (0.80) but low homogeneity (0.36). In other words, each variety is mostly kept together, but clusters mix several varieties. Table 3 and Figure 8b show that *k*-Means recovers a **size-driven grouping**: a very large cluster (81 of 81 Bombay beans), a small and round cluster (Dermason, Seker and most Sira; aspect ratio 1.43) and a medium-to-large, more elongated cluster (Horoz, Cali and Barbunya; aspect ratio 1.78). For reference only, *k* = 7 (the number of registered varieties) would give ARI 0.653 but a lower silhouette (0.303). The varieties overlap and are not separated by gaps, so internal criteria do not favour them.

Limitations of *k*-Means for this dataset and possible solutions:

- ***k* must be given, and the indices disagree** (2, 3 or 5). Possible solutions: stability-based selection, the gap statistic, domain knowledge, or a Gaussian mixture model with BIC.
- **Spherical, equal-variance assumption.** The bean classes are elongated, correlated ellipsoids of very different sizes and densities, and the 82 Bombay beans are far from the rest (Figure 8a). Possible solution: a Gaussian mixture with full covariance matrices, or clustering in a decorrelated (Mahalanobis or whitened) space.
- **Redundant features bias the distance.** About eight size variables (Area, Perimeter, both axes, ConvexArea, EquivDiameter, ...) give 'size' several times more weight than shape, so the clusters follow size. Possible solutions: feature selection, feature weighting or PCA (examined in 3.3B).
- **Sensitivity to initialisation and outliers.** The seed stability of a single run is only 0.84 (Table 5). Remedies: *k*-means++ with several restarts (used here), and robust scaling or *k*-medoids for outliers.

3.2 DBSCAN

Parameter choice. MinPts was set to **2 × dimensionality = 32** (Sander et al., 1998; Schubert et al., 2017). In 16 dimensions a larger MinPts gives more reliable density estimates and less sensitivity to noise. *eps* was taken from the ***k*-distance graph**, which sorts every bean's distance to its 32nd nearest neighbour (Figure 7a). Points left of the knee lie in dense regions and the steep tail contains sparse points. The knee, found with the same chord-distance rule, gives ***eps* = 2.48** (94.8th percentile). A sensitivity grid (Figure 7b) shows that the choice is not fragile: with MinPts = 32, every *eps* from 2.0 to 2.75 gives the same two-cluster solution with 1–3% noise, and neighbouring MinPts values give one or two clusters. Smaller *eps* (≤ 1.75) fragments the data and marks 8–53% of the beans as noise (MinPts ≥ 32), while *eps* = 3.0 merges almost everything into one cluster.

Results. DBSCAN found **2 clusters** (1,901 and 72 beans) and **27 noise points (1.4%)**. Cluster 1 contains only Bombay beans (72), while cluster 0 contains all six other varieties. The noise points are mainly extreme Horoz (12) and Bombay (8) beans.

Table 4. *k*-Means versus DBSCAN in the same standardised space (DB = Davies–Bouldin, CH = Calinski–Harabasz; internal metrics exclude DBSCAN noise; external metrics treat noise as its own group).

Algorithm	Clusters	Noise	Silhouette $\uparrow$	DB $\downarrow$	CH $\uparrow$	ARI $\uparrow$	NMI $\uparrow$
<i>k</i> -Means (<i>k</i> = 3)	3	0%	0.403	0.910	1,288	0.306	0.495
DBSCAN (<i>eps</i> = 2.48, MinPts = 32)	2	1.4%	0.554	0.529	557	0.034	0.158

Comparison and reasons for the differences. DBSCAN scores better on the separation indices (silhouette 0.554 vs 0.403; DB 0.529 vs 0.910) but far worse against the varieties (ARI 0.034 vs 0.306; NMI 0.158 vs 0.495). The silhouette advantage is not caused by removing noise, because *k*-Means scores 0.408 on the same non-noise beans. The algorithms differ for three reasons:

- **Different definitions of a cluster.** *k*-Means divides the space into *k* convex cells that minimise within-cluster variance, and it cuts a continuous mass whenever that lowers the error. It therefore splits the main body of beans into small versus medium/large beans (Figure 8b). DBSCAN only separates **density-connected** regions. The six non-Bombay varieties touch and overlap in one continuous dense cloud without low-density valleys (Figure 8a), so any *eps* large enough to connect a variety also chains it to its neighbours. Only Bombay is separated by a real empty gap.

- **What the internal metrics reward.** The 'Bombay versus rest' split is extremely well separated, so it scores well internally even though it carries little information about the varieties. This shows that internal indices alone can be misleading.
- **Strengths and weaknesses.** DBSCAN needs no k , flags outliers and can find arbitrary shapes. However, it uses one global density threshold (Dermason is dense while Bombay is sparse), and distance contrast shrinks in 16 dimensions, which makes ϵ hard to set. HDBSCAN or OPTICS, which allow varying densities, or clustering in a reduced space, are possible remedies.

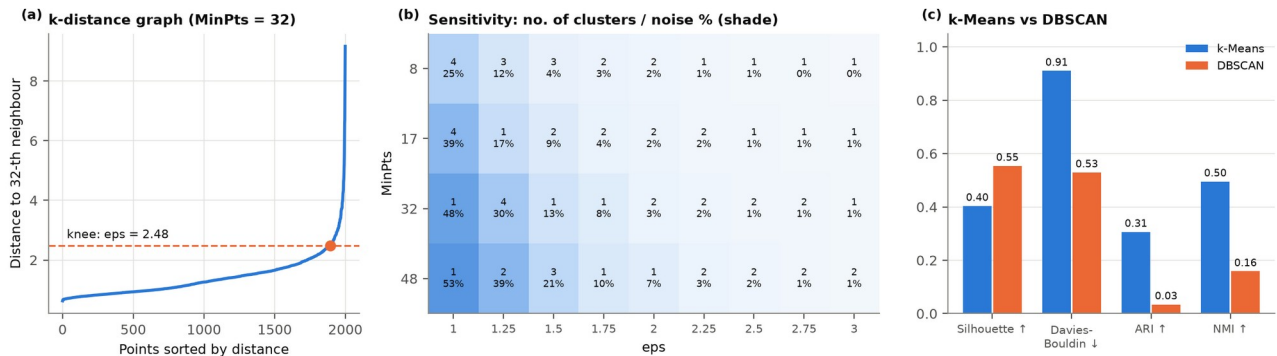


Figure 7. (a) Sorted distance to the 32nd nearest neighbour with the chosen ϵ at the knee; (b) number of clusters and noise percentage over a grid of ϵ and MinPts; (c) k-Means versus DBSCAN on the same internal and external metrics.

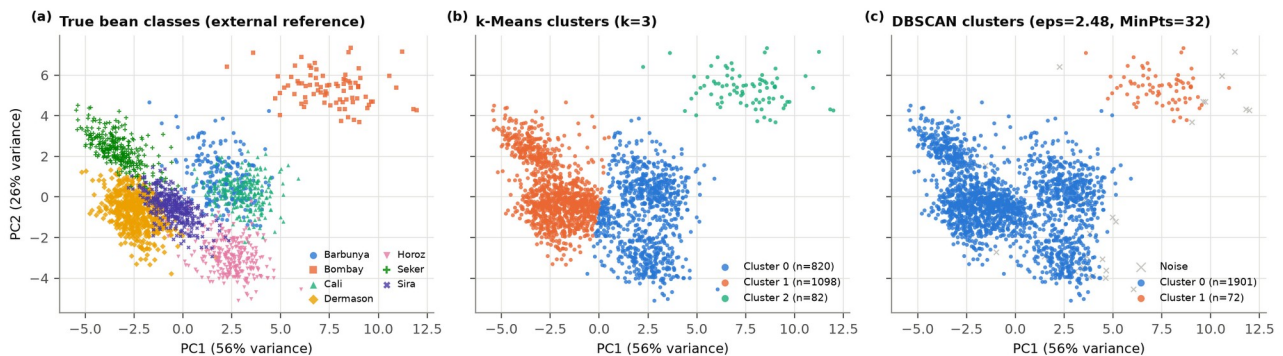


Figure 8. Beans projected onto the first two principal components (for visualisation only): (a) true classes, (b) k-Means clusters ($k=3$), (c) DBSCAN clusters (grey crosses = noise).

3.3B PCA Analysis

PCA was applied to the **standardised** inputs. On raw data the first component would simply reproduce Area, whose variance is about 10^{15} times larger than that of the shape factors.

Explained variance and the number of components. PC1 explains 55.6% of the variance (eigenvalue 8.91), PC2 26.0% (4.17), PC3 8.1% (1.29), PC4 5.2% (0.83) and PC5 2.9%, giving cumulative totals of 55.6, 81.7, 89.7, 94.9 and 97.8% (Figure 9a). Components 9–16 each explain less than 0.01%, because many variables are near-deterministic functions of others: $\text{EquivDiameter} = \sqrt{4 \cdot \text{Area} / \pi}$, and AspectRatio, Compactness and the shape factors are ratios of axes and area. The usual criteria give 3 components (Kaiser, eigenvalue > 1 , and the scree elbow), 4 (Jolliffe's relaxed cut-off of 0.7, and $\geq 90\%$ cumulative variance) or 5 ($\geq 95\%$). **Four components (94.9% of the variance)** were retained. This satisfies the 90% rule and Jolliffe's cut-off, which is preferred because the Kaiser rule tends to keep too few components when variables are highly correlated (Jolliffe, 2002). PC4 also represents a distinct, interpretable property, while PC5 adds only 2.9%.

Contribution of the variables (squared loadings, Figure 9b):

- **PC1, 'size versus roundness':** MajorAxisLength 10.6%, ShapeFactor2 9.9% (–), Perimeter 9.7% and EquivDiameter 8.9%. It is positive for all size and elongation variables and negative for roundness, compactness and the shape factors, so it separates large, elongated beans from small, round ones.
- **PC2, 'width versus elongation' (shape independent of size):** MinorAxisLength 11.6%, AspectRatio 11.4% (–), Compactness 11.3% and ShapeFactor3 11.2%.
- **PC3, 'outline regularity':** Solidity 54.6% and ShapeFactor4 26.1%. **PC4, 'bounding-box fill':** Extent 88.1%.

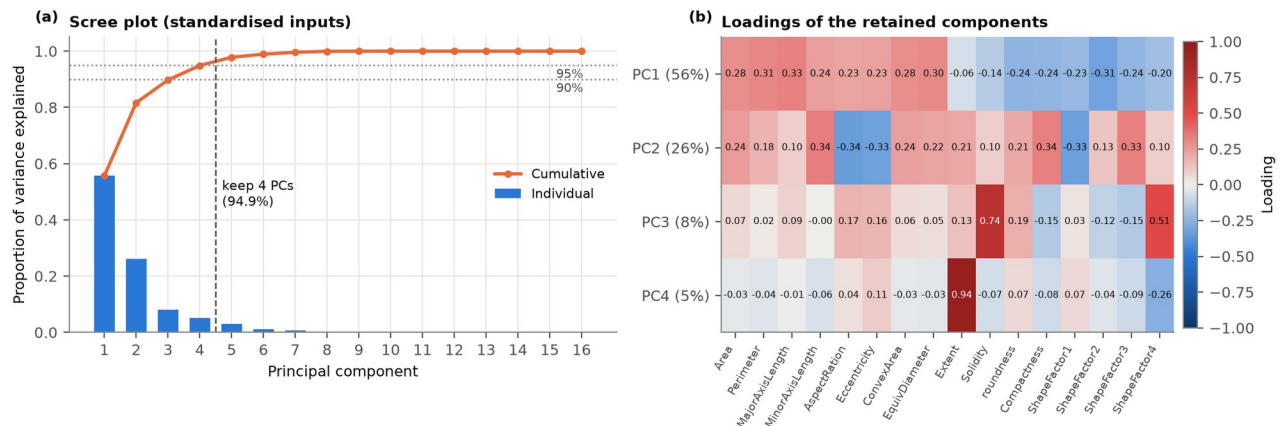


Figure 9. (a) Scree plot with individual and cumulative explained variance (dotted lines at 90% and 95%); (b) loadings of the 16 standardised variables on the four retained components.

Clustering in the PCA space. k-Means was applied to the four PC scores. The internal criteria were recomputed in this space and again select $k = 3$ (silhouette and DB; CH gives 2), so $k = 3$ was kept to allow a like-for-like comparison. A whitened version, with each PC rescaled to unit variance, was added to test the effect of re-weighting the components. Table 5 compares effectiveness, stability and cluster characteristics. Stability is measured in two ways. Bootstrap stability is the mean ARI between the full-data partition and partitions learnt on 20 random 80% subsamples. Seed stability is the mean pairwise ARI of 30 single-initialisation runs.

Table 5. k-Means ($k = 3$) on the original features versus PCA-transformed data.

Representation	Silh. (own)	Silh. (orig.)	DB	ARI	NMI	Agree. *	Boot. stab.	Seed stab.	Cluster sizes
Original (16 variables)	0.403	0.403	0.910	0.306	0.495	1.000	0.997	0.841	1098/820/82
PCA (4 PCs)	0.421	0.403	0.862	0.304	0.492	0.990	0.995	0.809	1099/819/82
Whitened PCA (4 PCs)	0.241	0.246	1.516	0.243	0.325	0.531	0.699	0.360	960/554/486

* Agree. = ARI between the partition in that space and the partition in the original space; Boot./Seed stab. = bootstrap and seed stability (mean ARI). Silh. (orig.) is the silhouette of the same partition measured in the original standardised space.

Impact of PCA. With four PCs, k-Means produces practically **the same clustering** as in the original space: the agreement ARI is 0.990, only 5 of 2,000 beans change cluster, and the cluster sizes and Bombay/small/medium profiles are the same. Effectiveness is unchanged (ARI 0.304 vs 0.306; NMI 0.492 vs 0.495) and so is stability (bootstrap 0.995 vs 0.997; seed stability 0.81 vs 0.84, a difference within its run-to-run variability, Figure 10c). Fitting was about $2.5\times$ faster (36 vs 91 ms). The only metrics that 'improve' are the internal ones computed in the reduced space (silhouette 0.421 vs 0.403; DB 0.862 vs 0.910). Figure 10a shows that this is an artefact. Silhouette in the own space grows steadily as components are dropped (0.65 with one PC), but when the same partitions are evaluated in the original space their silhouette stays at 0.40. Discarding low-variance directions shrinks the within-cluster distances; it does not make the clusters better separated.

Critical discussion: why PCA neither improves nor clearly deteriorates the clustering here.

- **Characteristics of k-Means.** k-Means uses Euclidean distance, which is unchanged by the orthogonal rotation PCA performs. Keeping six or more PCs reproduces the original partition exactly (agreement ARI = 1.000), so truncation can only remove information, here 5.1% of the variance. The size-driven cluster structure lies in PC1–PC2, and the leading principal components span the continuous solution of the k-means problem (Ding & He, 2004), so truncating to four PCs preserves the structure.
- **Feature correlation.** PCA removes the redundancy (16 correlated variables become 4 uncorrelated ones) but does not re-weight it. The redundant size variables re-appear as the dominant PC1 (56% of the variance), so k-Means in PCA space is still size-dominated and finds the same groups.
- **Trade-off between dimensionality reduction and information.** Four PCs keep 94.9% of the information with $4\times$ fewer dimensions and give the same clustering quality. The benefit is efficiency, a compact representation and 2-D visualisation (Figure 8), not effectiveness. Reducing further to one PC starts to lose structure (agreement 0.937, bootstrap stability 0.968).
- **Re-weighting the components hurts.** Whitening gives the low-variance components (Solidity, Extent), which mostly hold within-variety noise, the same weight as the size axis. The partition changes completely (agreement 0.53), ARI falls to 0.24 and the solution becomes unstable (seed stability 0.36). On this dataset the high-variance directions carry the discriminative structure.

- **The other algorithm.** Density-based clustering might be expected to benefit more, because distances are less concentrated in four dimensions. However, DBSCAN on the four PCs (MinPts = 8, knee eps = 1.44) again found only Bombay versus the rest (ARI 0.036). The overlap between varieties is real and is not caused by high dimensionality.

Conclusion. PCA compressed the 16 strongly correlated inputs into four interpretable components (size, shape, outline regularity, extent) with 95% of the information. Clustering on them is as effective, stable and similar in character as clustering on the original features, and it is cheaper. It is not more accurate, because k-Means already used this dominant variance structure. Getting clusters closer to the seven varieties would need a different distance weighting or cluster model (e.g. emphasising the shape components, or Gaussian mixtures), not dimensionality reduction alone.

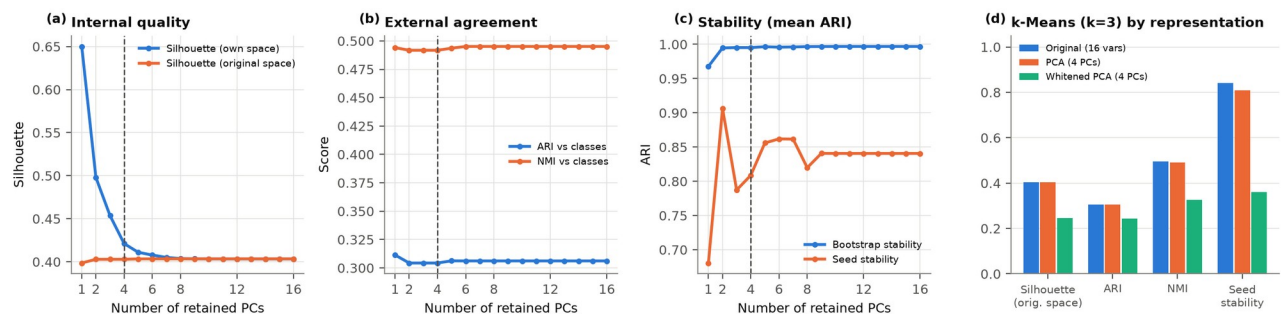


Figure 10. *k*-Means (*k* = 3) as a function of the number of retained PCs: (a) silhouette in the reduced versus the original space, (b) agreement with the true classes, (c) stability; (d) comparison of the original, PCA and whitened-PCA representations.

Acknowledgement of AI use

Claude (Anthropic, 2026), a generative AI tool, was used to help structure the analysis code, check the implementation and draft the wording of this report. All code was executed and all results, figures and statements were checked against the notebook outputs. The author takes responsibility for the content.

References

- Anthropic. (2026). *Claude* [Large language model]. <https://claude.ai/>
- Ding, C., & He, X. (2004). K-means clustering via principal component analysis. In *Proceedings of the Twenty-First International Conference on Machine Learning* (p. 29). ACM. <https://doi.org/10.1145/1015330.1015408>
- Gou, J., Zhan, Y., Rao, Y., Shen, X., Wang, X., & He, W. (2014). Improved pseudo nearest neighbor classification. *Knowledge-Based Systems*, 70, 361–375. <https://doi.org/10.1016/j.knosys.2014.07.020>
- Hubert, L., & Arabie, P. (1985). Comparing partitions. *Journal of Classification*, 2(1), 193–218. <https://doi.org/10.1007/BF01908075>
- Jolliffe, I. T. (2002). *Principal component analysis* (2nd ed.). Springer. <https://doi.org/10.1007/b98835>
- Koklu, M., & Ozkan, I. A. (2020). Multiclass classification of dry beans using computer vision and machine learning techniques. *Computers and Electronics in Agriculture*, 174, Article 105507. <https://doi.org/10.1016/j.compag.2020.105507>
- Mitani, Y., & Hamamoto, Y. (2006). A local mean-based nonparametric classifier. *Pattern Recognition Letters*, 27(10), 1151–1159. <https://doi.org/10.1016/j.patrec.2005.12.016>
- Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, É. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.
- Rousseeuw, P. J. (1987). Silhouettes: A graphical aid to the interpretation and validation of cluster analysis. *Journal of Computational and Applied Mathematics*, 20, 53–65. [https://doi.org/10.1016/0377-0427\(87\)90125-7](https://doi.org/10.1016/0377-0427(87)90125-7)
- Sander, J., Ester, M., Kriegel, H.-P., & Xu, X. (1998). Density-based clustering in spatial databases: The algorithm GDBSCAN and its applications. *Data Mining and Knowledge Discovery*, 2(2), 169–194. <https://doi.org/10.1023/A:1009745219419>
- Satopää, V., Albrecht, J., Irwin, D., & Raghavan, B. (2011). Finding a "kneedle" in a haystack: Detecting knee points in system behavior. In *2011 31st International Conference on Distributed Computing Systems Workshops* (pp. 166–171). IEEE. <https://doi.org/10.1109/ICDCSW.2011.20>
- Schubert, E., Sander, J., Ester, M., Kriegel, H.-P., & Xu, X. (2017). DBSCAN revisited, revisited: Why and how you should (still) use DBSCAN. *ACM Transactions on Database Systems*, 42(3), Article 19. <https://doi.org/10.1145/3068335>